PROJECT CONTEXT BRIEF

PR Reviewer Agent

A multi-agent, evidence-verified pull request reviewer for GitHub — specialist findings are proposed by parallel Claude Agent SDK sessions and independently re-derived by a fresh-context checker before anything is reported.

● Working MVP — pipeline validated end-to-end against a live, real-world PR

This document gives full context on what the project is, why it's built the way it is, and how its pieces fit together — for anyone picking it up without prior conversation history. It complements `README.md` (quick start) and `docs/DOCUMENTATION.md` (module-level technical reference).

VERSION	RUNTIME	ENGINE	PREPARED
0.1.0	Node.js 18+	Claude Agent SDK	19 Aug 2026

01 The Problem

Most AI PR reviewers run a single pass over a diff and narrate confidence — "this looks fine," "this might be an issue" — with no independent check on whether the claim is actually true. That produces two failure modes at once: plausible-sounding false positives that erode trust, and real issues missed because one pass only looks at the diff text, not the live repository.

02 Design Philosophy

> **Maker ≠ checker.** Eight specialist agents propose candidate findings independently and never see each other's output. A separate agent, started with zero shared context, re-derives every claim from the repository before it counts.

> **Repository is the source of truth.** Every agent reads the actual checked-out PR branch — `Read/Grep/Glob/Bash` — not just the diff text. A finding without evidence the agent actually read is rejected.

> **Gates are computed, not narrated.** Evidence checks, duplicate merges, and the final risk verdict are decided in plain deterministic code — never asserted by an LLM.

> **Small and high-confidence beats exhaustive.** Every agent is instructed to prefer zero findings over a weak one.

03 Review Pipeline

1. **Fetch & checkout**
   PR metadata, diff, and changed files pulled via the GitHub CLI; the PR branch is cloned into a disposable temp directory (or an already-checked-out local path is reused).

2. **Discovery agent**
   One session detects languages, frameworks, architecture, the PR's actual intent, and project invariants — each backed by a concrete source in the repo, merged into persistent state.

3. **8 maker agents, in parallel**
   Bug Hunter, Security, Performance, Architecture, Testing, API/Database, Framework Specialist, DevOps/CI — each a fresh session scoped to one concern, returning structured JSON findings.

4. **Mechanical gates**
   Deterministic code rejects findings with no evidence and merges cross-agent duplicates by file, overlapping lines, and title similarity — before any verify tokens are spent.

5 **Fresh-context VERIFY agent**

A brand-new session, with no memory of the makers' reasoning, independently re-checks every surviving candidate against the repo and diff — applying 7 explicit gates and returning VALID / INVALID / UNCERTAIN.

6 **Real verification commands**

Detects and runs the smallest applicable check — `npm test`, `dart analyze`, `go test ./...`, `pytest` — so the report carries actual exit-code evidence.

7 **Verdict, report, persistence**

Risk and verdict are computed in code from verified severities; a no-tool-access synthesis pass writes only the prose summary; the report is checkpointed and printed, saved, and/or posted to the PR.

04 Agent Roster

AGENT	SCOPE
Discovery	Languages, frameworks, architecture, PR intent, evidence-backed project invariants.
Bug Hunter	Conditions, null handling, races, state transitions, error handling, resource lifecycle.
Security	AuthN/AuthZ, injection, SSRF, secrets, unsafe deserialization — traced attack surface → entry point → data flow → impact.
Performance	Algorithmic complexity, N+1 queries, blocking operations, unbounded concurrency/memory, missing timeouts.
Architecture	Layer boundaries and dependency direction — only where it creates measurable engineering risk.
Testing	What changed, what could regress, what's actually covered — reads real test files, doesn't assume.
API / Database	HTTP semantics and validation; SQL/Mongo index and query-pattern checks.
Framework Specialist	Detects Flutter / Node / React usage from the repo itself, applies only the relevant checklist.
DevOps / CI	Dockerfiles, GitHub Actions, migrations, rollback safety — things that break deploys even when tests pass.
Verify	Fresh-context checker — re-derives every candidate finding from the repo, applies GATE1–GATE7, returns a verdict.
Synthesis	No repo access — writes only the Summary and Positive Changes prose, strictly grounded in already-verified data.

05 How It Runs

CLI

A local Node.js command against any PR URL or repo/number pair.

- Print, save (`--out`), or post (`--post`) the report
- Review an already-checked-out path with `--local-dir`
- Reuses the last cached report for an unchanged commit

GitHub Action

A template workflow to drop into any repo you want reviewed automatically.

- Triggers on PR opened / synchronized / reopened
- Posts the report as a PR comment via `GITHUB_TOKEN`
- Requires an `ANTHROPIC_API_KEY` repo secret

06 Real-World Validation

MEDIUM — VERIFIED

genesis-kit PR #3 — "fix argument parsing"

The pipeline was run against a real open PR with no prior tuning for this repo. Of 5 candidate findings from the 8 maker agents, the VERIFY agent rejected 3 as unsupported and confirmed 2. The surviving MEDIUM finding was a genuine, reproducible bug: the fix made the PR's own documented invocation succeed, but a variable computed *before* the new argument-shift logic runs still captured a flag token as the project name — reproduced with an actual `bash -x` trace and a one-line suggested fix, not a guess.

AGENTS RUN	CANDIDATES	VERIFIED	REJECTED	CACHE RE-RUN
10	5	2	3	~1.5S

07 Project Status

- [x] Full pipeline implemented and passing a live end-to-end run
- [x] Structured, schema-enforced agent output (no free-text parsing)
- [x] Deterministic gates, risk calc, and report formatting
- [x] Persistent invariants / history / checkpoint cache
- [x] CLI with print / save / post / local-dir / force / cache
- [ ] Not yet pushed to a GitHub remote or committed
- [ ] Action template needs `REVIEWER_REPO` filled in after publishing
- [ ] No unit tests yet for gates / risk-calc / report modules

- GitHub-only for now — GitLab/Bitbucket adapters not built
- No inline per-line PR comments yet (summary comment only)

08 Repository Map

```
bin/pr-review.js          # CLI entry point
src/orchestrator.js       # pipeline: wires every stage together
src/github/gh.js          # all GitHub I/O via the gh CLI
src/agents/runAgent.js    # one Claude Agent SDK session, start to finish
src/schemas.js            # JSON Schema contracts for every agent
src/prompts/              # discovery / makers / verify / synthesis / shared
src/gates.js              # mechanical evidence + duplicate gates
src/riskCalc.js           # verdict computed from verified severities
src/report/format.js      # deterministic markdown report
src/state/store.js        # invariants / history / checkpoints on disk
src/verify/runCommands.js # npm test / dart analyze / go test / pytest
.github/workflows/pr-review.yml # GitHub Action template
docs/DOCUMENTATION.md     # module-level technical reference
```